



Παράλληλα Συστήματα
Εργασία Pthreads

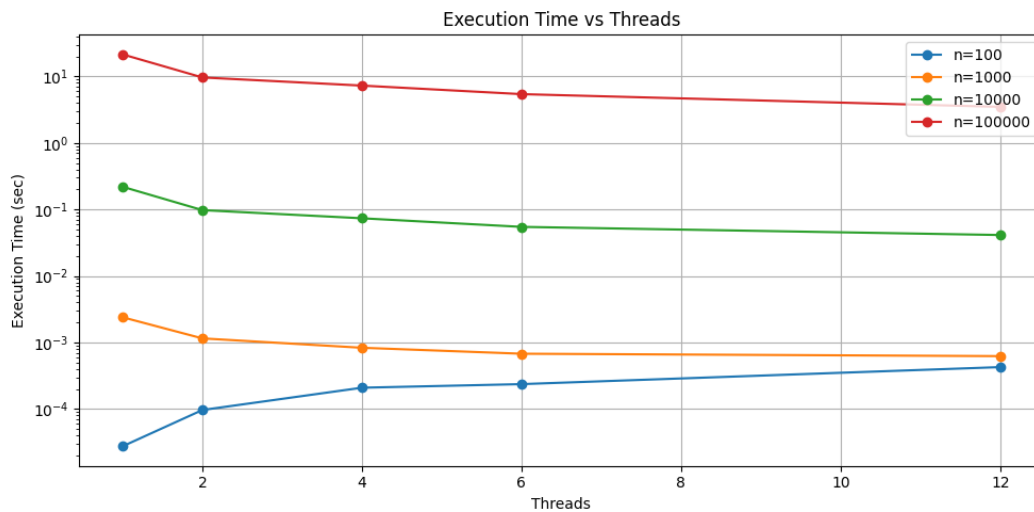
Ονοματεπώνυμο: Ευάγγελος Τραϊκόγλου

Αριθμός Μητρώου: 1115202200189

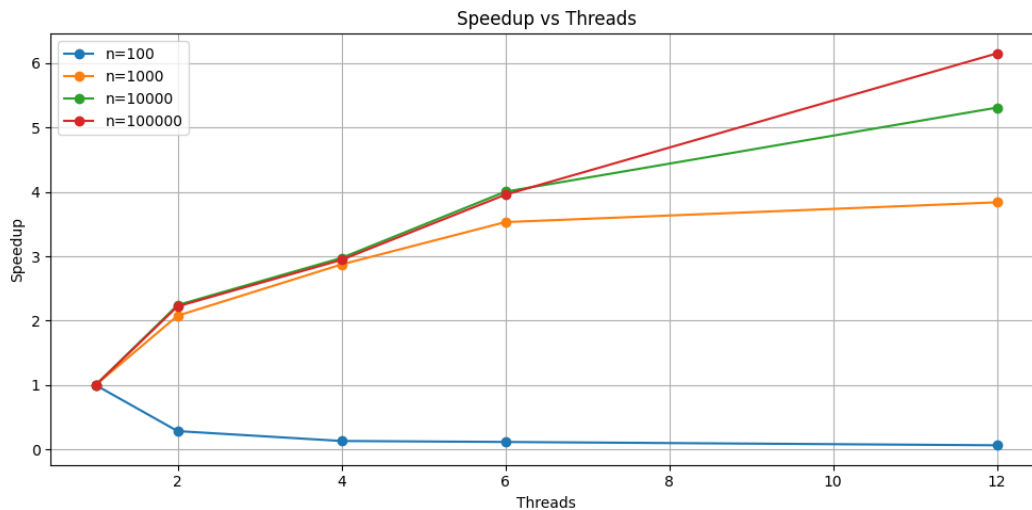
- cpu: i7-8700
- cores: 12 logical, 6 physical
- os, kernel: Ubuntu 24.04.3 LTS, 6.8.0-87-generic
- comp version: gcc 13.3.0

1 Polynomial Multiplication

Η πρώτη άσκηση επικεντρώνεται στον πολλαπλασιασμό πολυωνύμων βαθμού n . Ο σειριακός αλγόριθμος ακολουθεί την απλή υλοποίηση $O(n^2)$ χρόνου. Στον παράλληλο, κάθε νήμα υπολογίζει ένα εύρος του διανύσματος αποτελέσματος μεγέθους $\lceil (2n+1)/nthreads \rceil$, σύμφωνα με τον τύπο: $res_k = \sum_{i=0}^k a_i \cdot b_{k-i}$. Αυτό επιτρέπει τη μη χρήση συγχρονισμού αφού κάθε νήμα γράφει σε διαφορετικές θέσεις. Και οι δύο (για βαθμό ή φορτίο ≥ 1024) χρησιμοποιούν εκδοχή με blocking (bs = 128), αφού πετυχαίνεται καλύτερη απόδοση από τον βαθμό αυτό και μετά.



Σχήμα 1: Execution time vs threads for polynomial multiplication

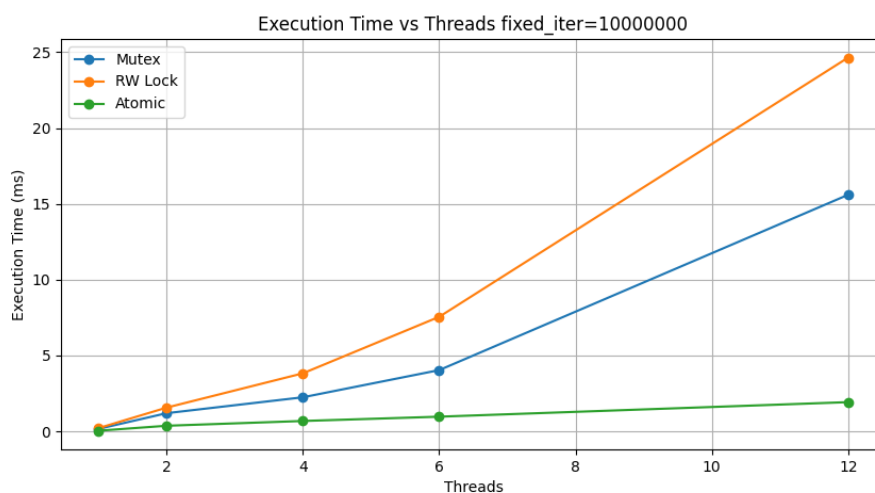


Σχήμα 2: Speedup vs threads for polynomial multiplication

Συμπεράσματα: Για μικρά n (πχ 100), η αύξηση των νημάτων αυξάνει τον χρόνο εκτέλεσης αφού επηρεάζεται κυρίως από την ύπαρξή τους. Κάθως όμως το n αυξάνεται, η αύξηση των νημάτων επιφέρει μείωση του χρόνου εκτέλεσης. Βέβαια το κέρδος από την εισαγωγή ενός νέου νήματος μειώνεται, ακολουθώντας τον νόμο της φθίνουσας απόδοσης. Αντίστοιχα, η επιτάχυνση (με βάση το σειριακό) μεγιστοποιείται στον μέγιστο αριθμό από νήματα και για μεγαλύτερες τιμές του n .

2 Update Shared Var

Η δεύτερη άσκηση επικεντρώνεται στον πιο αποδοτικό τρόπο ανανέωσης μιας κοινόχρηστης μεταβλητής. Η υλοποίηση με `atomic` εντολές χρησιμοποιεί το όρισμα `_ATOMIC_RELAXED` αφού δε χρειάζεται κάποια διάταξη όσο αναφορά τις αναφορές στη μνήμη πριν ή μετά την `atomic` εντολή.



Σχήμα 3: Execution time vs threads (update shared var)



Σχήμα 4: Execution time vs iterations (update shared var)

All Runs

```

1 method,threads,iterations,avg_time_ms
2 m,1,10000,0.00038826750000000004
3 m,1,100000,0.00192572675
4 m,1,1000000,0.016825314
5 m,1,10000000,0.16162775699999998
6 m,2,10000,0.00124869425000000001
7 m,2,100000,0.01076761575
8 m,2,1000000,0.111890217250000001
9 m,2,10000000,1.19802298725
10 m,4,10000,0.0022988365
11 m,4,100000,0.02759573925
12 m,4,1000000,0.22508347075
13 m,4,10000000,2.24688139425
14 m,6,10000,0.00474706775
15 m,6,100000,0.04782153575
16 m,6,1000000,0.41482762149999997
17 m,6,10000000,4.03888313375
18 m,12,10000,0.014560627
19 m,12,100000,0.15107985275
20 m,12,1000000,1.51693118475
21 m,12,10000000,15.59940501575
22 rw,1,10000,0.00035414775
23 rw,1,100000,0.002553964
24 rw,1,1000000,0.02256181025
25 rw,1,10000000,0.21220591
26 rw,2,10000,0.00172123925000000001
27 rw,2,100000,0.015409842
28 rw,2,1000000,0.147941219
29 rw,2,10000000,1.56544050325
30 rw,4,10000,0.0038236777499999998
31 rw,4,100000,0.0388237155
32 rw,4,1000000,0.3817210055
33 rw,4,10000000,3.8274963435
34 rw,6,10000,0.0076861922499999995

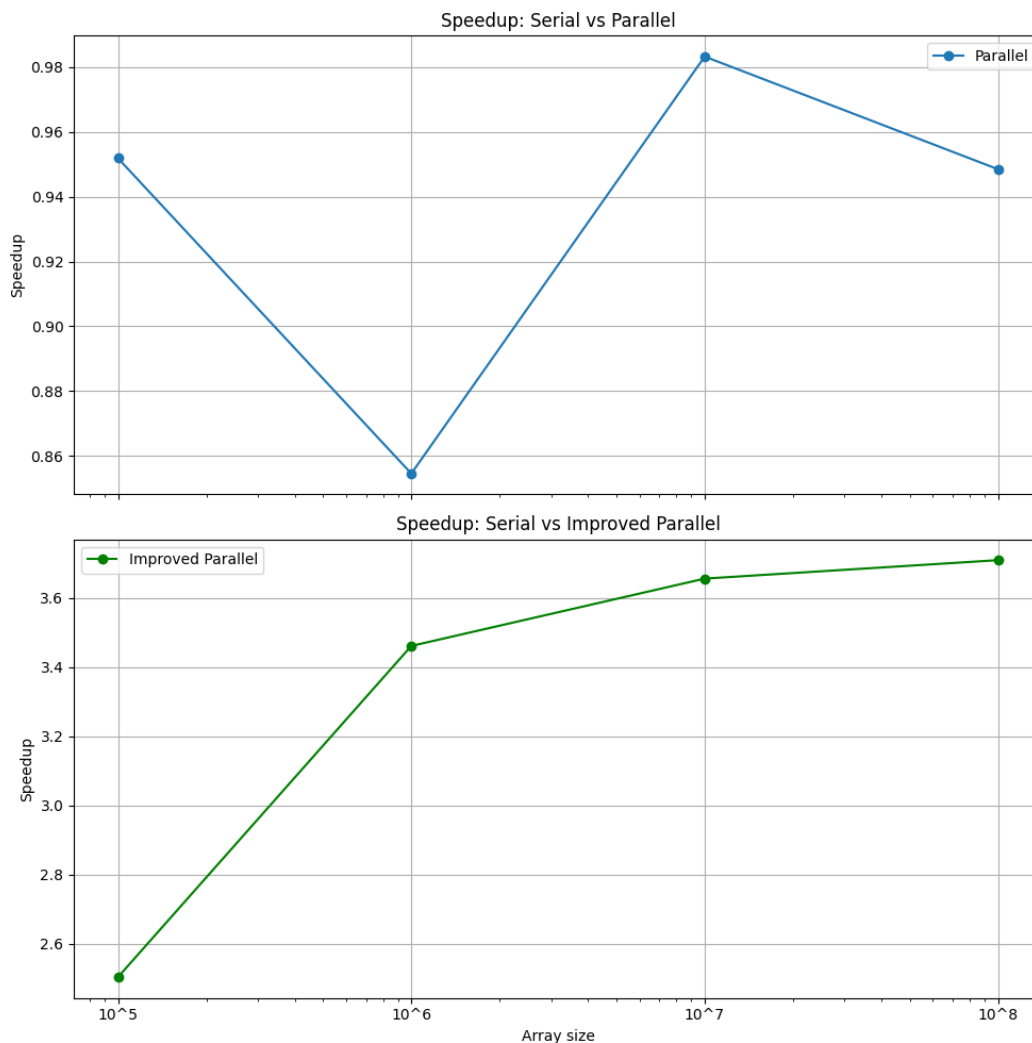
```

```
35 rw,6,100000,0.0757629335
36 rw,6,1000000,0.742311727
37 rw,6,10000000,7.5534937145
38 rw,12,10000,0.023157905
39 rw,12,100000,0.23958616775
40 rw,12,1000000,2.42496255775
41 rw,12,10000000,24.64496008975
42 a,1,10000,0.00017844125
43 a,1,100000,0.00056038625
44 a,1,1000000,0.0043706835
45 a,1,10000000,0.04190325875
46 a,2,10000,0.000380574
47 a,2,100000,0.003154238
48 a,2,1000000,0.026213148749999998
49 a,2,10000000,0.3687871685
50 a,4,10000,0.000704333
51 a,4,100000,0.00703517425
52 a,4,1000000,0.0693055845
53 a,4,10000000,0.68362061975
54 a,6,10000,0.0009918729999999999
55 a,6,100000,0.010478943000000001
56 a,6,1000000,0.09870726925000001
57 a,6,10000000,0.9669541875
58 a,12,10000,0.0021271325
59 a,12,100000,0.01968003225
60 a,12,1000000,0.17763742575
61 a,12,10000000,1.92574091625
```

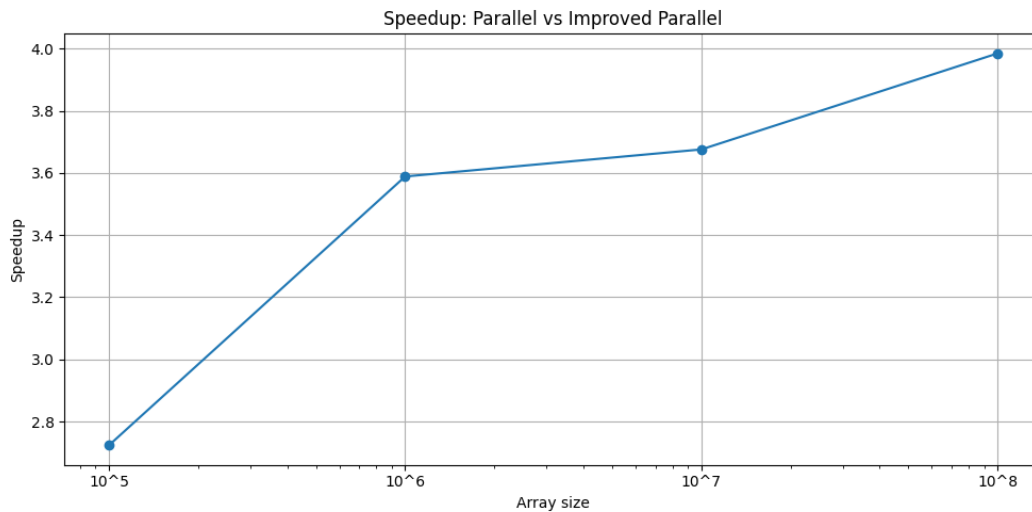
Συμπεράσματα: Η υλοποίηση με την atomic εντολή είναι η πιο γρήγορη για όλα τα πειράματα που έγιναν, μετά τα mutexes και τέλος τα rwlocks. Αυτό είναι λογικό αφού οι atomic εντολές υποστηρίζονται από το hardware. Μεταξύ των mutexes και rwlocks, ο επιπρόσθετος χρόνος των 2ων είναι γιατί έχουν πιο σύνθετη υλοποίηση, λύνοντας το readers-writers πρόβλημα. Όταν όμως χρησιμοποιούνται σαν mutexes (όπως εδώ), επιβαρύνουν τον χρόνο εκτέλεσης.

3 Table Stats

Η τρίτη άσκηση αφορά τη καταμέτρηση των μη μηδενικών στοιχείων 4 πινάκων σε 4 νήματα σε μια κοινή δομή. Ανάλογα αν έχει οριστεί η IMPROVED χρησιμοποιείται ή όχι η βελτιωμένη εκδοχή, όπου το struct είναι padded για να γεμίζει ένα cache line ο κάθε μετρητής, για να αποφευχθεί το φαινόμενο του false sharing.



Σχήμα 5: Speedup: Serial vs Parallel



Σχήμα 6: Speedup: Parallel vs improved

Συμπεράσματα: Αρχικά, δεν χρειάζεται συγχρονισμός αφού κάθε νήμα γράφει σε διαφορετική μεταβλητή. Ο απείριστος παράλληλος αλγόριθμος δεν επιτυγχάνει καμία βελτίωση έναντι του σειριακού. Αυτό δείχνει πόσο το false sharing επηρεάζει την απόδοση. Η βελτιωμένη εκδοχή όπου το κάθε cache line έχει έναν μετρητή πλησιάζει την ιδανική επιτάχυνση (x4) καθώς το n αυξάνεται.

4 Bank

Η τέταρτη άσκηση προσομοιώνει τη λειτουργία μιας τράπεζας. Κάθε νήμα κάνει ένα μείγμα συναλλαγών (μεταφοράς χρημάτων, ερώτησης υπολοίπου). Χρησιμοποιούνται δύο είδη κλειδομάτων (mutexes, rwlocks). Το ζητούμενο είναι να μελετηθεί η επίδραση της επιλογής των κλειδομάτων στις διάφορες επιλογές παραμέτρων. Υλοποιήθηκαν wrapper functions lock,unlock (Προφανώς η επιλογή READ,WRITE στη lock έχει νόημα μόνο για τη περίπτωση των rwlocks).

thread_work: Δύο επιλογές: fine grain: 1 lock ανά λογαριασμό, coarse grain: 1 lock για τον πίνακα. Κάθε νήμα κάνει perc% ερωτήσεις υπολοίπου και 1-perc% μεταφορές χρημάτων. Προφανώς στις μεταφορές πρέπει να ελέγχεται αν αυτός που στέλνει έχει υπόλοιπο ≥ 0 . Το μόνο write lock που χρειάζεται είναι στην ανανέωση του υπολοίπου του παραλήπτη.

Benchmark file: csv/bank_results.csv

```
1 Array Size,Transactions/Thread,Query %,Threads,Lock Type,Lock Granularity,Avg Parallel Time
2 100,100000,10,6,m,f,2.16299435625
3 100,100000,10,6,m,c,2.16487088725
4 100,100000,10,6,rw,f,2.16273627725
5 100,100000,10,6,rw,c,2.1702455665000002
6 -----
7 100,100000,50,6,m,f,1.69253447775
8 100,100000,50,6,m,c,1.6830908535
9 100,100000,50,6,rw,f,1.696206595
10 100,100000,50,6,rw,c,1.68470176625
```

Συμπεράσματα: Η ύπαρξη μεγαλύτερου ποσοστού μεταφορών (μεγαλύτερο κρίσιμο τμήμα) δημιουργεί περισσότερα collisions στο coarse grain locking, άρα το fine grain υπερτερεί σε αυτή τη περίπτωση. Τα rwlocks πλησιάζουν τα mutexes όταν το ποσοστό της ερώτησης υπολοίπου είναι υψηλό και παρατηρούνται αρκετά collisions σε λογαρισμούς, καθώς αυτά επιτρέπουν ταυτόχρονο read lock. Καθώς μεγαλώνει το μέγεθος του πίνακα, η fine grain λογική συμφέρει.

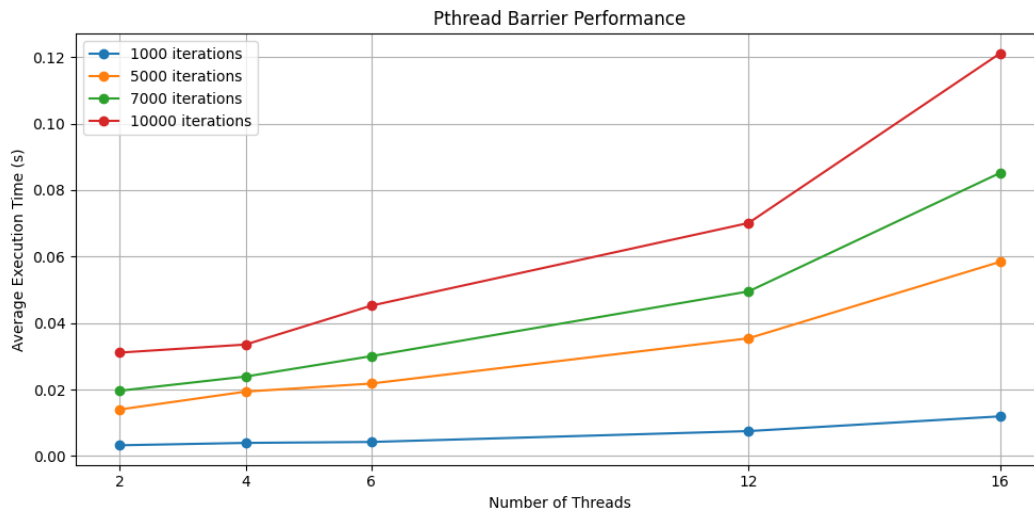
CS = 100 microseconds

```
1 Array Size,Transactions/Thread,Query %,Threads,Lock Type,Lock Granularity,Avg Parallel Time
2 100,100,50,6,m,f,0.01024307575
3 100,100,50,6,m,c,0.05409800175
4 100,100,50,6,rw,f,0.01268990725
5 100,100,50,6,rw,c,0.07664481125
6 -----
7 100,100,90,6,m,f,0.017782965749999997
8 100,100,90,6,m,c,0.09606632175
9 100,100,90,6,rw,f,0.01762629275
10 100,100,90,6,rw,c,0.068098498000000001
11 -----
12 100,1000,50,6,m,f,0.12159310675
13 100,1000,50,6,m,c,0.555413368
14 100,1000,50,6,rw,f,0.11336553775
15 100,1000,50,6,rw,c,0.7481444775
16 -----
17 100,1000,90,6,m,f,0.1703641175
18 100,1000,90,6,m,c,0.9165540555
19 100,1000,90,6,rw,f,0.15749289125
20 100,1000,90,6,rw,c,0.645123842
21 -----
22 100,10000,50,6,m,f,1.132857148
23 100,10000,50,6,m,c,5.50221585375
24 100,10000,50,6,rw,f,1.0579327455
25 100,10000,50,6,rw,c,7.33189534025
26 -----
27 100,10000,90,6,m,f,1.6689945987500001
28 100,10000,90,6,m,c,9.16758695275
29 100,10000,90,6,rw,f,1.61094170575
30 100,10000,90,6,rw,c,6.50010087
31 -----
32 100,100000,50,6,m,f,10.64145038025
33 100,100000,50,6,m,c,55.12728169675
34 100,100000,50,6,rw,f,10.281757949
35 100,100000,50,6,rw,c,73.144967487
36 -----
37 100,100000,90,6,m,f,16.3196365285
38 100,100000,90,6,m,c,91.46850930275
39 100,100000,90,6,rw,f,15.826938326499999
40 100,100000,90,6,rw,c,64.27206109125001
```

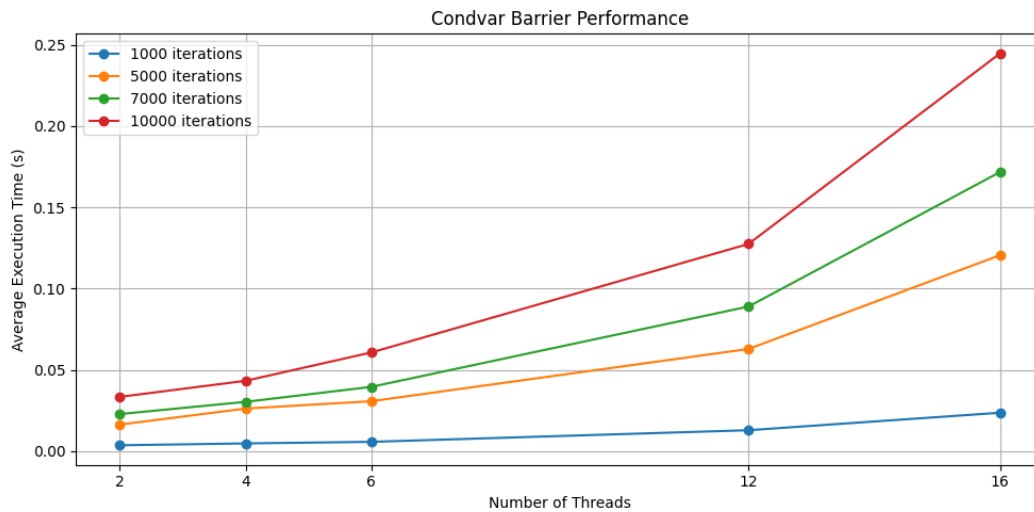
Συμπεράσματα: Έχοντας μεγαλύτερο κρίσιμο τμήμα στην ερώτηση υπολοίπου, παρατηρείται πως καθώς αυξάνεται η δουλειά ανά νήμα, σε συνδυασμούς με υψηλό ποσοστό ερώτησης υπολοίπου, τα rwlocks αποδίδουν καλύτερα από τα mutexes, λόγω της δυνατότητας του ταυτόχρονου read lock που χρειάζεται σε αυτή τη περίπτωση (αφού τα collisions ανά λογαριασμό αυξάνονται).

5 Barrier

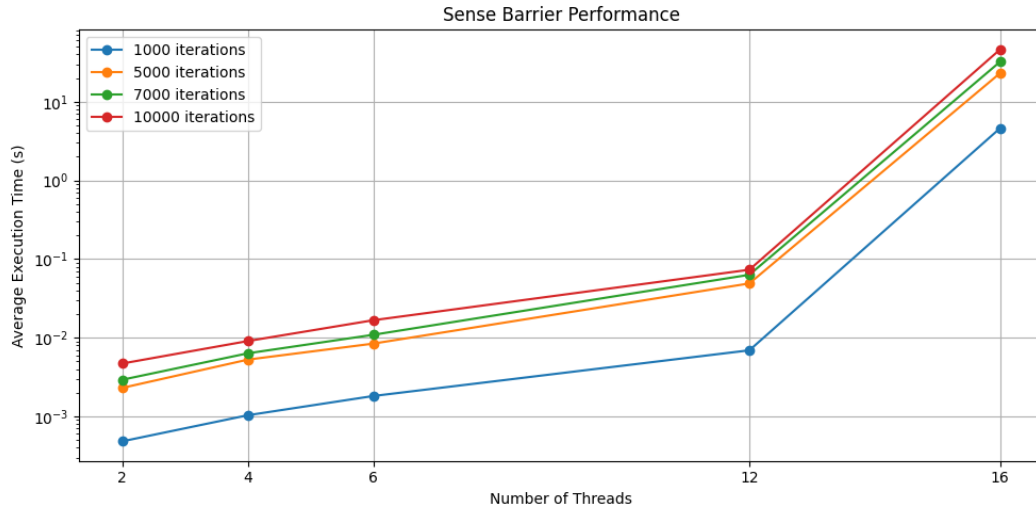
Η πέμπτη άσκηση χρησιμοποιεί τρεις διαφορετικές υλοποιήσεις barrier για ένα loop, pthread,cond var,sense barrier. Για τις υλοποιήσεις: Pthread $\Rightarrow$ pthread lib, Cond var $\Rightarrow$ παράδειγμα βιβλίου, Sense $\Rightarrow$ wikipedia.



Σχήμα 7: Pthread Barrier



Σχήμα 8: Cond Var Barrier



Σχήμα 9: Sense Barrier

Barrier Benchmark

```

1 Iterations,Threads,Pthread,Condvar,Sense
2 1000,2,0.00325200575,0.00364375025,0.0004837385
3 1000,4,0.0039657775,0.00479563625,0.0010367155
4 1000,6,0.00425344275,0.0057634735,0.0018192537499999999
5 1000,12,0.007521949,0.0129179515,0.006928470000000001
6 1000,16,0.0119462095,0.023657546749999998,4.608054978249999
7 -----
8 5000,2,0.014019862000000001,0.01638969375,0.002309737
9 5000,4,0.019396135999999998,0.026231251,0.00527501
10 5000,6,0.021821416750000003,0.03075606025,0.008424714
11 5000,12,0.03543104425,0.06290929175,0.049018785499999995
12 5000,16,0.058430050000000004,0.12059134425000001,23.07544673425
13 -----
14 7000,2,0.0196623595,0.0228532625,0.00293217025
15 7000,4,0.023928938,0.030372813250000002,0.0063464135
16 7000,6,0.030063930250000002,0.03959025975,0.010905662000000002
17 7000,12,0.0495310365,0.08901890075000002,0.063139517
18 7000,16,0.08516762675,0.1717566115,32.20471212875
19 -----
20 10000,2,0.031135567,0.03342347925,0.00471107175
21 10000,4,0.0335539475,0.043314635500000004,0.0090838975
22 10000,6,0.0452556005,0.06075280675,0.01670283625
23 10000,12,0.0701135625,0.12753812474999998,0.07318805349999999
24 10000,16,0.12112650575,0.2447712185,46.16105747575

```

Συμπεράσματα: Όσο ο αριθμός των νημάτων είναι ≤ 12 , δηλαδή $\leq$ λογικών πυρήνων, ο sense υπερτερεί καθώς τα νήματα περιμένουν σε while loop και δεν κοιμούνται όπως στις άλλες 2 υλοποιήσεις. Όταν όμως ο αριθμός των νημάτων γίνει 16, ο sense χειροτερεύει σε επίδοση αρκετά (μερικά δευτερόλεπτα έναντι κάτω του 1sec στις άλλες 2) καθώς κάθε πυρήνας απασχολείται με την εκτέλεση άσκοπων εντολών (το while loop) μην αφήνοντας την πρόοδο των νημάτων που δεν έχουν φτάσει ακόμα στο barrier. Προφανώς η υλοποίηση της Pthread είναι καλύτερη από την απλή υλοποίηση με μεταβλητές συνθήκης (futexes, άλλες βελτιστοποιήσεις για τη συγκεκριμένη αρχιτεκτονική...). Το sense είναι επαναχρησιμοποιούμενο επειδή κάθε νήμα παρακολουθεί ένα τοπικό flag και το global flag αντιστρέφεται μόνο όταν όλα τα νήματα ολοκληρώσουν 1 φάση.